

Interim Status Report

FreqEasy-ML, Machine Learning-Based Acoustic Room Assessment Assistant

Kyle Dick (ID: 7206287)

Spring 2026 – Level 11

Bachelor of Applied Engineering – Electronic Systems Engineering, Conestoga
College

ENGG7420 – Advanced Technical Elective I

Professor Dr. Scott Chen

July 21, 2026

Table of Contents

1 INTRODUCTION.....	4
2 ENGINEERING INFORMATION AND DOCUMENTATION	5
3 RESEARCH AND DESIGN INVESTIGATION.....	6
3.1 Machine Learning Platform Investigation.....	6
3.2 Classification Model Investigation.....	7
3.3 Input Feature Investigation.....	8
<i>Final node-level input features</i>	<i>9</i>
<i>Four-node fused features.....</i>	<i>9</i>
<i>Features excluded from model training.....</i>	<i>10</i>
3.4 Classification-Label Investigation.....	11
<i>Final label definitions.....</i>	<i>12</i>
<i>Recommended future classification structure</i>	<i>13</i>
4 EXPERIMENTAL INVESTIGATIONS	14
4.1 <i>Experimental Objectives and Setup.....</i>	<i>14</i>
4.2 <i>FFT Frequency-Detection Results.....</i>	<i>15</i>
4.3 <i>Data-Pipeline and Interface Verification</i>	<i>15</i>
4.4 <i>Engineering Decisions From Testing.....</i>	<i>16</i>
4.5 <i>Room-Condition Dataset and Preliminary Classification Results</i>	<i>16</i>
5 Engineering Design Progress	17
5.1 Quality and Completeness.....	17
5.2 Design Documentation.....	18
5.2.1 <i>Overall System Architecture</i>	<i>18</i>
5.2.2 <i>Recording Node Data Flow</i>	<i>19</i>
5.2.3 <i>Server and Analysis Data Flow.....</i>	<i>19</i>
5.2.4 <i>Improvement Over the Proposal Design</i>	<i>20</i>
6 ENGINEERING TOOLS.....	20
7 APPLICATIONS OF KNOWLEDGE AND NEW LEARNING	20
8 PROJECT MANAGEMENT AND CURRENT STATUS.....	21
8.1 Project Schedule.....	21
8.2 Risks and Mitigation.....	21
<i>Table 8.2: Updated Project Risk Register.....</i>	<i>21</i>
9 REMAINING WORK	22

9.1 Dataset Completion and Validation.....	22
9.2 ANN Training and Evaluation	24
9.3 Recommendation and Interface Integration	25
10 CONCLUSIONS	25
11 REFERENCES	26
APPENDICES	26
Appendix A: 4 Node Dataset Instructions	26
Appendix B: Logbooks & Status Reports.....	33
Appendix C: Feature Dictionary for Dataset 3.....	42

ACKNOWLEDGEMENTS

This project is possible due to the School of Engineering and Technology at Conestoga College, specifically the faculty of the Electronic Systems Engineering – Bachelor of Applied Engineering Degree program.

1 INTRODUCTION

FreqEasy-ML is a machine learning-based acoustic assessment system intended to help non-expert users interpret the frequency behaviour of small rooms. It extends the Smart Distributed Acoustic Calibration System (SDACS) by using acoustic features collected from four distributed sensor nodes as inputs to a supervised classification model.

The system is designed to identify broad acoustic conditions, including:

- Low-frequency emphasis
- Mid-frequency emphasis
- High-frequency emphasis
- Speech
- Environmental noise
- Quiet-room conditions

Rather than presenting only FFT plots and numerical measurements, FreqEasy-ML is intended to convert the classification result into understandable guidance. Possible recommendations include:

- Investigating bass buildup
- Reviewing speaker placement
- Considering absorptive treatment
- Repeating a measurement under quieter conditions
- Reporting that no strong acoustic condition was detected

These recommendations are intended as decision-support guidance and not as certified acoustic-design conclusions.

My individual work during this phase focused on:

- Assisting with defining the machine-learning workflow
- Preparing and validating labelled datasets
- Assisting with the investigation of Edge Impulse deployment
- Selecting acoustic input features for the dataset
- Removing invalid or label-leaking fields
- Comparing individual-node and multi-node data
- Documenting the Raspberry Pi and Flutter integration path

The project has progressed beyond the proposal and preliminary dataset stages. A balanced six-class dataset has now been collected from all four SDACS nodes, cleaned, and divided into capture-level training and testing sets.

The latest dataset contains:

- 11,426 raw node-level rows

- 11,040 cleaned node-level rows
- Six balanced labels
- 1,840 cleaned rows per label
- 2,760 four-node fused samples
- 460 fused samples per label

A preliminary model comparison showed that combining synchronized measurements from all four nodes provided substantially better classification performance than treating each node independently. The next phase is to train and evaluate the final feedforward ANN in Edge Impulse, deploy the selected model on the Raspberry Pi, and display its output through the SDACS interface.

2 ENGINEERING INFORMATION AND DOCUMENTATION

Table 1: Project Development Activities

Date	Activity	Evidence produced	Outcome
June 23, 2026	Completed project proposal	Proposal document	Project scope and objectives established
June 30, 2026	Investigated Edge Impulse CSV requirements	Research notes and sample CSV files	Initial data format and label structure selected
July 2, 2026	Prepared initial FFT dataset	Labelled CSV files	Dataset import workflow established
July 7, 2026	Trained initial classifier	Confusion matrix and model results	Preliminary classification performance evaluated
July 8–14, 2026	Collected four-node test-tone data	Node-RED exports and MATLAB handoff files	Frequency-feature behaviour compared across nodes
July 14, 2026	Completed first Edge Impulse deployment library	Initial model library	Raspberry Pi deployment workflow verified
July 19, 2026	Collected six-label room-condition dataset	Two synchronized four-node CSV exports	Approximately eight minutes collected for each label
July 19–20, 2026	Corrected and cleaned the latest dataset	Cleaning audit, feature dictionary, training and testing CSVs	Balanced Edge Impulse-ready dataset produced
July 20, 2026	Completed preliminary multi-node comparison	Random Forest results and confusion matrices	Four-node fusion selected as the preferred modelling approach

Additional evidence is provided in:

- Individual logbooks (Appendices)

- Weekly project status reports (Appendices)
- Node-RED flow exports
- Dataset-cleaning records
- MATLAB handoff files
- Edge Impulse project files
- Backend and Flutter integration tests
- Project appendices and supporting ZIP folders

3 RESEARCH AND DESIGN INVESTIGATION

3.1 Machine Learning Platform Investigation

Several machine-learning platforms were evaluated for FreqEasy-ML:

- Edge Impulse
- Python with scikit-learn
- MATLAB Classification Learner
- TensorFlow or LiteRT

The platforms were compared based on their ability to:

- Import tabular acoustic data
- Manage labels and datasets
- Train classification models
- Display confusion matrices
- Support model comparison
- Export a deployable model
- Integrate with the Raspberry Pi
- Support possible future embedded deployment

Edge Impulse was selected as the primary model-development platform. It provides a structured workflow for:

- Uploading labelled CSV data
- Separating training and testing data
- Configuring a feedforward ANN
- Reviewing validation results
- Generating confusion matrices
- Measuring model size and inference time
- Exporting a Raspberry Pi-compatible model

The initial deployment target is the Raspberry Pi 5. Direct execution on the ESP32-S3 remains a possible future improvement after the feature-based server model has been fully validated.

MATLAB remains part of the SDACS analysis workflow. Its primary uses are:

- FFT visualization

- Node comparison
- Frequency-band plotting
- PCA or clustering analysis
- Three-dimensional acoustic-room mapping
- Calibration analysis

TensorFlow and LiteRT provide greater flexibility for custom neural networks and embedded deployment. However, they require more implementation and optimization effort and are not required for the current minimum viable system.

Table 2: Comparison of Machine-Learning Development Platforms

Platform	Main advantages	Main limitations	Intended use in FreqEasy-ML
Edge Impulse	Guided dataset, training, testing, confusion-matrix, deployment, and model-profiling workflow	Less control over custom preprocessing than a fully coded workflow	Primary ANN development and Raspberry Pi model export
Python and scikit-learn	Flexible cleaning, feature selection, node fusion, model comparison, and independent validation	Requires custom scripts and manual workflow management	Dataset preparation and preliminary sanity checks
MATLAB Classification Learner	Strong signal-processing, plotting, calibration, and exploratory-analysis tools	Additional work is required for direct backend deployment	Acoustic visualization and engineering analysis
TensorFlow or LiteRT	Supports custom neural networks and future embedded inference	Greater programming, optimization, and deployment complexity	Future raw-audio or ESP32-S3 model development

Current development status includes:

- An Edge Impulse project has been created.
- An initial model library has been tested on the Raspberry Pi.
- The latest six-label dataset has been prepared for import.
- Four-node fused training and testing CSVs have been generated.
- Final ANN training and Raspberry Pi deployment remain to be completed.

3.2 Classification Model Investigation

FreqEasy-ML uses a non-recurrent feedforward artificial neural network. The network evaluates a fixed acoustic feature vector without relying on previous states or feedback loops.

This structure was selected because:

- The firmware produces one-second feature summaries.

- Each input row represents one acoustic measurement interval.
- The current model does not process continuous raw-audio sequences.
- The input is tabular rather than an audio waveform.
- A small dense network is suitable for Raspberry Pi deployment.

The preferred model input is the four-node fused dataset. Each fused sample represents one synchronized second of room measurements.

For each acoustic feature, the system calculates:

- Mean across the four nodes
- Standard deviation across the four nodes
- Range across the four nodes

This produces a room-level feature vector rather than four independent node predictions.

A preliminary Random Forest model was used only as an independent dataset-quality check.

Table 3: Preliminary Dataset Comparison

Dataset approach	Samples	Approximate capture-held-out accuracy	Interpretation
Individual-node data	11,040	23.8%	Strong node and class variability
Four-node fused data	2,760	47.2%	Distributed sensing provides improved class separation

These are not final Edge Impulse ANN results. However, they support the decision to use room-level fusion for the primary classifier.

3.3 Input Feature Investigation

The acoustic features are produced by the SDACS nodes before transmission to the Raspberry Pi.

Each node uses:

- ESP32-S3 microcontroller
- ICS-43432 digital I²S microphone
- 48 kHz sampling rate
- 24-bit audio data
- 2,048-point FFT
- Approximately one-second feature windows

The feature-based approach reduces storage and transmission requirements because the system does not need to continuously transfer raw PCM audio.

Final node-level input features

The cleaned dataset uses 19 acoustic features:

- dBFS
- Acoustic peak frequency
- Low-rumble ratio
- Bass-band ratio
- Low-mid-band ratio
- Mid-band ratio
- Presence-band ratio
- High-band ratio
- Bass-band peak frequency
- Low-mid-band peak frequency
- Mid-band peak frequency
- Presence-band peak frequency
- High-band peak frequency
- Dominant-band ratio
- Dominant-band peak frequency
- Broad FFT low-frequency ratio
- Broad FFT mid-frequency ratio
- Broad FFT high-frequency ratio
- Total FFT energy

The firmware groups spectral measurements into the following functional regions:

- Sub-frequency region
- Bass
- Low-mid
- Mid
- Presence
- High

The exact frequency limits are defined by the current firmware implementation and should be included in the firmware appendix or feature dictionary.

Four-node fused features

The room-level dataset calculates three statistics for each of the 19 features:

- Mean
- Standard deviation
- Range

This produces:

19 acoustic features × 3 fusion statistics = 57 model inputs

Features excluded from model training

Several columns were retained only as metadata or removed entirely.

The following were excluded because they could reveal the correct label:

- target_band
- target_freq_hz
- suggested_z_metric

The following were excluded because they describe the collection process rather than the acoustic condition:

- Node ID
- Run ID
- Timestamp
- Sequence number
- Capture ID
- Firmware version

Temperature and humidity were recorded as contextual metadata but were not used as model inputs. This prevents the model from learning the collection time or environmental drift instead of the acoustic class.

Estimated SPL was also excluded from the final feature set because it has not yet been calibrated against a reference sound-level meter. The calibrated result may be reintroduced in a later model.

Redundant or constant fields were removed, including:

- Constant sub-band fields
- Duplicate energy measurements
- RMS where its information was already represented by dBFS
- Estimated SPL where it duplicated dBFS with an offset
- Calibration-only detector fields

The exported CSV retains the original numerical feature scales. Any normalization used during ANN training must also be applied identically during Raspberry Pi inference.

See a complete sample row and feature dictionary included in the appendix.

Table 4: Final Acoustic Feature Selection

Feature group	Purpose	Final use
---------------	---------	-----------

Full FFT-bin amplitudes	Provide detailed spectral information	Not included in the current reduced feature model
Frequency-band ratios	Describe energy distribution across the spectrum	Included
Band peak frequencies	Identify strong frequencies within each region	Included
Dominant peak and band	Identify the strongest spectral region	Included
Broad low/mid/high FFT ratios	Summarize overall spectral balance	Included
dBFS	Represent relative digital acoustic level	Included
RMS	Represent signal energy	Excluded because of redundancy with dBFS
Estimated SPL	Provide an understandable physical level	Excluded until calibration is completed
FFT total energy	Represent overall spectral energy	Included
Temperature and humidity	Provide environmental context	Metadata only
Node ID and capture information	Support traceability and debugging	Metadata only
Target frequency or target band	Describe the selected test condition	Excluded because of label leakage
Raw audio	Preserve temporal acoustic information	Future-work feature source

3.4 Classification-Label Investigation

The original proposal considered several possible outputs:

- Low-frequency buildup
- Mid-frequency emphasis
- High-frequency emphasis
- Flat or balanced response
- Excessive background noise
- Speech presence
- Reflection-related issues

Testing showed that these outputs describe different acoustic properties and may overlap. For example:

- Speech contains substantial mid-frequency energy.
- Environmental noise may contain low-, mid-, and high-frequency components.
- Cymbals contain high-frequency energy but may also be broadband.

- Quiet-room recordings may still contain HVAC or computer-fan noise.
- A room may contain speech and low-frequency emphasis at the same time.

Despite this overlap, a six-class structure was selected for the first full room-condition dataset so that the complete workflow could be tested.

The exact labels are:

- low
- mid
- high
- speech
- noisy
- quiet_room_white_noise

Final label definitions

Low

- Bass guitar
- Kick drum
- 808 or low-frequency synthesizer
- Other recordings dominated by low-frequency energy

Mid

- Piano
- Guitar
- Female vocals

High

- Cymbals
- Hi-hats
- Flute
- Violin Harmonics
- Piccolo

Speech

- Spoken voice
- Podcasts
- Male or female speech
- Conversation

Noisy

- Crowd noise
- Café ambience
- Ceiling Fan
- Microwave
- Mixed environmental noise

Quiet-room/white-noise

- Empty-room background
- HVAC or fan noise
- Low-activity room conditions
- White-noise examples included in the current dataset

Each label contains:

- 1,840 cleaned node-level samples
- 460 room-level fused samples
- Four logical captures
- Three training captures
- One testing capture

Table 5: Final Classification Labels

Edge Impulse label	Measurement condition	Node-level samples	Fused samples	Current role
low	Low-frequency instruments and bass-heavy material	1,840	460	Primary class
mid	Mid-range instruments and musical content	1,840	460	Primary class
high	Cymbals and high-frequency source material	1,840	460	Primary class
speech	Spoken voice and conversation	1,840	460	Primary class
noisy	Crowd, café, appliances, and mixed noise	1,840	460	Primary class
quiet_room_white_noise	Quiet-room conditions and white-noise examples	1,840	460	Primary class with known limitations

Recommended future classification structure

The latest results suggest that a two-stage or multi-output design may be more technically appropriate.

Table 6: Recommended Future Classification Structure

Output category	Possible labels	Purpose
Spectral condition	Low dominant, mid dominant, high dominant	Describe the distribution of acoustic energy
Room scene	Quiet room, speech, noisy	Describe the type of room activity
Measurement quality	Acceptable, unreliable	Determine whether the capture should be used
Treatment guidance	Investigate bass buildup, review absorption, repeat measurement, no major issue detected	Convert results into user-facing guidance

This structure would allow more than one condition to be reported at the same time. For example:

- Speech present
- Mid-frequency dominant
- Acceptable measurement quality

Treatment recommendations should remain separate from the machine-learning labels. They should be generated using rule-based logic after the classifier output has been reviewed.

Reflection-related recommendations remain future work because the current dataset contains FFT-derived summary features rather than full impulse-response, decay-time, or time-domain measurements.

4 EXPERIMENTAL INVESTIGATIONS

4.1 Experimental Objectives and Setup

Experimental testing evaluated whether SDACS could provide reliable data for the FreqEasy-ML classifier. The investigation focused on FFT peak-frequency detection, four-node acoustic data collection, system communication via MQTT and Raspberry Pi, local microSD storage, FastAPI-to-Flutter data consistency, and the collection of labelled machine-learning data.

Four ESP32-S3 acoustic nodes with ICS-43432 digital I²S microphones were tested using controlled low-, mid-, and high-frequency tones. The nodes operated at a 48 kHz sampling rate with 24-bit audio data and were positioned close together to maintain similar conditions.

Table 4.1: FFT Frequency-Detection Test Conditions

Frequency range	Test tones	Capture duration	Nodes tested
-----------------	------------	------------------	--------------

Low	100 Hz, 500 Hz	20 s per tone	Four
Mid	2 kHz, 5 kHz	20 s per tone	Four
High	10 kHz, 15 kHz	20 s per tone	Four

Firmware testing confirmed that the nodes could successfully connect to Wi-Fi and the MQTT broker, initialize the I²S microphones at 48 kHz, acquire acoustic and environmental data, publish measurements, and store files locally on the microSD card.

One controlled test produced an estimated, uncalibrated LAeq value of 91.60 dB, a peak level of 94.00 dB, and a temperature of approximately 25.2 °C.

4.2 FFT Frequency-Detection Results

The four nodes showed similar frequency-detection behaviour. Strong agreement was observed at 5 kHz and 15 kHz, while greater variation occurred between 100 Hz and 2 kHz, with the largest inconsistency near the 2 kHz test point. The similarity of error patterns across nodes suggests a systematic limitation rather than individual hardware faults.

Table 4.2: Summary of Frequency-Detection Performance

Test region	Observed performance	Effect on FreqEasy-ML
100–500 Hz	Significant variation and reduced accuracy	Review, calibrate, or exclude unreliable values
Around 2 kHz	Inconsistent peak detection	Treat the boundary region cautiously
5 kHz	Close agreement with the applied tone	Suitable for model input
10–15 kHz	Strong agreement with expected frequency	Suitable for model input

Lower-frequency inaccuracies may be influenced by microphone response, loudspeaker output, FFT peak-selection behaviour, room reflections, and sensor placement. Further calibration is required to isolate these effects.

These findings impact the machine-learning workflow, as uncorrected low-frequency data may cause the ANN to learn system limitations rather than room characteristics. Such inputs should therefore be calibrated or treated cautiously.

4.3 Data-Pipeline and Interface Verification

Backend-to-frontend testing confirmed that SDACS telemetry could be transferred from the Raspberry Pi FastAPI backend to Flutter without altering its meaning.

The backend used:

```
/api/nodes/{node_id}/features/latest
```

The test involved requesting the latest node record, inspecting the backend JSON, viewing the same node in Flutter, and comparing all values.

Table 4.3: Backend-to-Frontend Data Consistency

Field	Backend value	Flutter value	Result
Node ID	node04	node04	Match
SPL	32.99 dB	32.99 dB	Match
Peak frequency	46.9 Hz	46.9 Hz	Match
Temperature	24.87 °C	24.9 °C	Match after rounding
Relative humidity	46.77%	46.8%	Match after rounding
Battery level	98.13%	98%	Match after rounding
Firmware version	ota-enable-v2	ota-enable-v2	Match

Differences were limited to expected rounding, confirming consistency across the data pipeline.

4.4 Engineering Decisions From Testing

Testing confirmed that SDACS can reliably collect, store, transmit, and display acoustic and environmental data, as well as generate datasets for MATLAB and Edge Impulse.

Engineering decisions included using consistent labels for ANN training, treating SPL values as approximate until calibration is complete, and reviewing measurements below 2 kHz before drawing conclusions. Calibration corrections and frequency-band ratios will be incorporated where appropriate, while avoiding metadata that could bias classification. Final testing will use complete unseen capture sessions, and bass-treatment recommendations will remain limited until low-frequency performance is validated.

4.5 Room-Condition Dataset and Preliminary Classification Results

A larger dataset was collected using four synchronized nodes to represent realistic acoustic conditions. The six labels used were Low, Mid, High, Speech, Noisy, and Quiet-room/white-noise, with approximately eight minutes of data collected per label.

Data cleaning involved correcting mislabeled sessions, restructuring quiet-room recordings, removing invalid records, trimming capture edges, validating feature integrity, and ensuring proper dataset splitting using complete capture sessions.

Table 4.4: Cleaned Room-Condition Dataset

Dataset characteristic	Result
Raw feature rows	11,426
Cleaned node-level rows	11,040
Classes	6
Samples per class	1,840

Nodes represented	4
Training rows	8,280
Testing rows	2,760
Split method	Complete capture session

Three captures per label were used for training, with one reserved for testing to prevent data leakage.

A room-level dataset was created by combining node data using mean, standard deviation, and range across one-second intervals, producing 2,760 fused samples.

Table 4.5: Preliminary Classification Sanity Check

Dataset approach	Approximate accuracy
Individual-node samples	23.8%
Four-node fused samples	47.2%

Although based on a preliminary Random Forest model, the results indicate that multi-node fusion improves classification performance.

Limitations include overlap between speech and mid-frequency data, mixed-frequency content in noisy recordings, variability between nodes, and the loss of temporal detail due to one-second feature summarization. Future development will focus on improving calibration, increasing data diversity, refining label separation, and applying stable prediction averaging across full capture sessions.

5 Engineering Design Progress

The following table shows the progress towards the final integration of the ANN model and recommendation logic on flutter:

5.1 Quality and Completeness

Project requirement	Current status	Evidence / remaining work
Collect data from SDACS nodes	Complete	Data collected from all four ESP32-S3 nodes
Extract acoustic features	Complete	FFT, RMS, dBFS, estimated SPL, and frequency-band data exported
Create labelled datasets	In progress	Test-tone dataset prepared; room-condition dataset version 1 complete

Train ANN classifier	In progress	Initial Edge Impulse model created and imported on the RPi; final multi-node model still need
Validate frequency measurements	Partially complete	Mid- and high-frequency results were accurate with some overlap with speech label; low-frequency results require further review
Integrate backend and interface	Mostly complete	Raspberry Pi, FastAPI, and Flutter data consistency verified, additional flutter cards / widgets needed
Generate treatment recommendations	In progress	Final Recommendation logic not currently defined or fully validated
Complete final system testing	Not complete	End-to-end ANN classification and recommendation testing still to be done

Overall, the major hardware, firmware, communication, and data-collection components are operational. The main remaining work is to improve the room-condition dataset, complete ANN evaluation, and validate the final user recommendations.

5.2 Design Documentation

5.2.1 Overall System Architecture

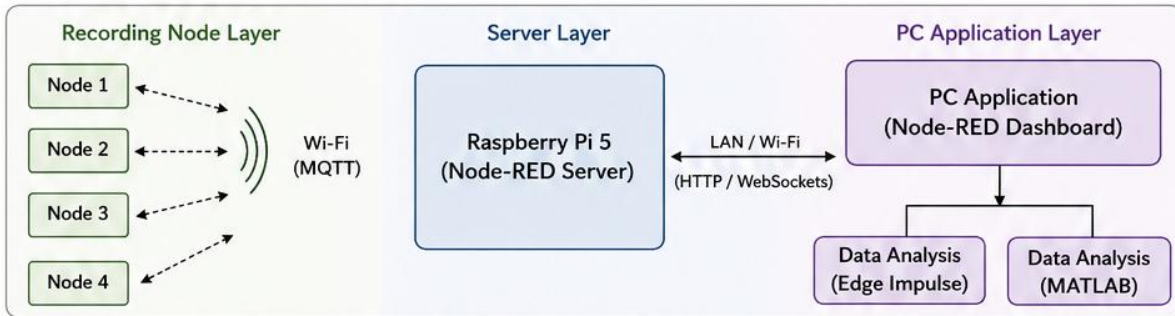


Figure 1: SDACS Conceptual System Architecture

Figure 1 divides SDACS into three main layers: the distributed recording nodes, the Raspberry Pi 5 server, and the PC application layer. Four ESP32-S3 nodes collect room measurements and publish telemetry through Wi-Fi using MQTT. The Raspberry Pi runs Node-RED and coordinates data reception, storage, and system control. The PC application displays the measurements and provides access to the Edge Impulse and MATLAB analysis tools.

This design separates sensing, processing, and user interaction. As a result, the recording nodes can remain focused on data collection while the server performs storage, validation, and communication with the analysis applications.

5.2.2 Recording Node Data Flow

Figure 2 shows the internal data flow of one SDACS recording node. The MEMS microphone captures the room's acoustic signal, while the SHT41 sensor measures temperature and humidity. These digital measurements are processed by the ESP32-S3 microcontroller, which calculates acoustic features such as FFT data, SPL, RMS, dBFS, and RT60-related values.

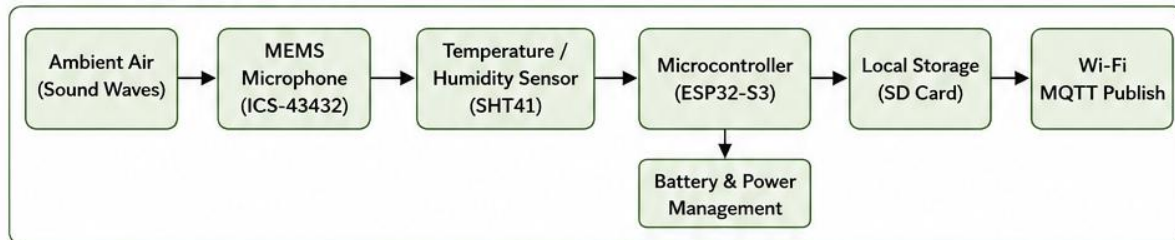


Figure 2: SDACS Recording Node Data Flow

The processed measurements are stored locally on the SD card and transmitted to the server through MQTT. Battery monitoring and power-management functions provide battery-status information for the node. Local storage also protects the measurement data if the wireless connection is temporarily unavailable.

5.2.3 Server and Analysis Data Flow

Figure 3 shows how the Raspberry Pi receives and processes data from the recording nodes. MQTT messages enter the Node-RED server, where they are parsed and checked for valid node identifiers, timestamps, and measurement fields. The verified data is then stored or buffered for dashboard display and later analysis.

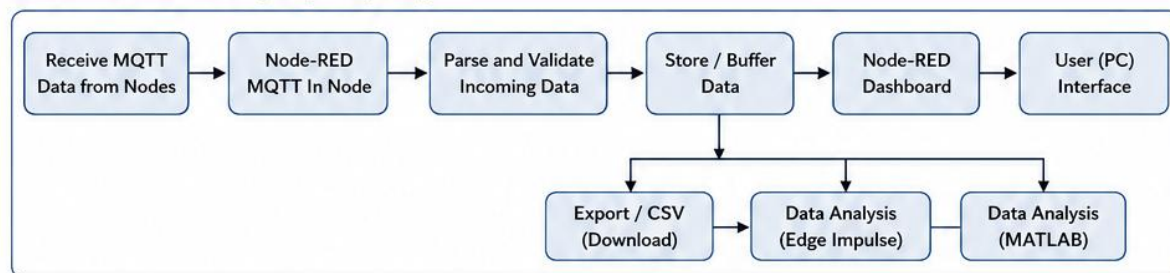


Figure 3: SDACS Server Data Flow

The user can view live node information through the Node-RED or Flutter interface and download completed measurements as CSV files. These CSV files provide the connection between SDACS and FreqEasy-ML. Validated FFT-derived features can be imported into Edge Impulse for ANN training and classification, while MATLAB can be used for additional acoustic analysis and visualization. Backend testing has confirmed that node telemetry can be transferred from the Raspberry Pi to the user interface without changing the recorded values.

5.2.4 Improvement Over the Proposal Design

These diagrams expand the original proposal's simplified **CSV → Edge Impulse → output** block diagram. The updated documentation shows the complete path from room measurement and embedded processing to MQTT communication, server validation, CSV storage, ANN analysis, and user-facing results. This provides a more complete description of how FreqEasy-ML integrates with the existing SDACS system.

6 ENGINEERING TOOLS

Several engineering tools were used across the FreqEasy-ML and SDACS workflow.

Tool	Purpose
ESP-IDF	Developed and tested ESP32-S3 firmware for audio capture, FFT processing, sensor readings, MQTT communication, and SD-card logging.
Node-RED and MQTT	Coordinated node captures, monitored node status, received telemetry, and stored CSV/SQLite data on the Raspberry Pi.
MATLAB and Excel	Reviewed CSV data, plotted FFT results, and compared target and measured frequencies.
Edge Impulse	Managed labelled datasets and trained the non-recurrent ANN classifier.
FastAPI and Flutter	Transferred Raspberry Pi data to the user interface and displayed live node measurements.
GitHub	Managed firmware and application versions using branches and shared repositories.
Draw.io	Created system, node, server, and data-flow diagrams.

These tools support the complete process from acoustic measurement and embedded processing to data storage, ANN classification, and user-facing results.

7 APPLICATIONS OF KNOWLEDGE AND NEW LEARNING

The development of FreqEasy-ML required applying knowledge from embedded systems, digital signal processing, communication networks, and software integration. Previous work on SDACS provided experience with ESP32-S3 firmware, I²S microphone acquisition, FFT processing, MQTT communication, Raspberry Pi services, and sensor-data logging.. Finally, collecting audio samples in specific classes was a new experience.

Important new learning during this phase included:

- Preparing FFT and sensor data for machine learning.
- Defining clear and repeatable classification labels in Node-RED.
- Identifying the effect of poor low-frequency measurements on ANN training.
- Verifying data consistency between the recording nodes, backend, and Flutter interface.

The main technical challenge was ensuring that the ANN learns actual acoustic patterns rather than limitations of the measurement hardware. Testing showed that the SDACS nodes detect mid- and high-frequency tones more accurately than frequencies below approximately 2 kHz. This led to a better understanding of dataset quality, calibration, feature selection, and the risk of training a model using biased measurements. Future work will apply this learning by improving the dataset, testing the selected ANN, and validating whether its classifications produce useful and responsible acoustic-treatment recommendations.

8 PROJECT MANAGEMENT AND CURRENT STATUS

8.1 Project Schedule

The following Gantt chart shows the progression of AI ML components of SDACS so far:

Table 8.1: FreqEasy-ML Project Gantt Chart

Project Activity	Jun. 22	Jun. 29	Jul. 6	Jul. 13	Jul. 20	Jul. 27	Aug. 3	Aug. 10	Current Status
Proposal and literature review	Done	--	--	--	--	--	--	--	Done
SDACS experimental-data review	START								3 rd Run
FFT frequency validation		START							MB Done
CSV dataset preparation	START		DONE						Labelling
Edge Impulse project setup	START		Lib#1			Lib#2			Lib#1 Done
Recommendation-logic development					START			DONE	STARTED
Backend and Flutter integration	START					DONE			Finalizing
Final validation and documentation				START				DONE	STARTED

8.2 Risks and Mitigation

The original SDACS risks were reviewed and updated to include risks specific to FreqEasy-ML dataset preparation and ANN development.

Table 8.2: Updated Project Risk Register

Risk	Likelihood	Impact	Mitigation or Current Response
------	------------	--------	--------------------------------

Insufficient measured data	High	High	Collect repeated measurements and supplement them with controlled synthetic data where appropriate.
Inconsistent low-frequency FFT measurements	High	High	Apply calibration, flag unreliable samples, or limit early training to the validated frequency range.
Node synchronization	low	High	Use synchronized group capture, timestamps, and validation checks before combining node records.
Edge Impulse data-format limitations	Moderate	Moderate	Standardize CSV columns and test small datasets before uploading the full dataset.
Scope creep	Moderate	Moderate	Prioritize the minimum viable option in gui + tinyML
Time constraints	High	High	Prioritize dataset validation, ANN training, and GUI functionality before MATLAB graphs.
Budget constraints	Low	Moderate	Reuse the four SDACS nodes, Raspberry Pi, speaker, and available test equipment.
Battery handling and safety	Low	Low	Follow low-voltage and ESD procedures and inspect batteries before testing.

9 REMAINING WORK

9.1 Dataset Completion and Validation

Three datasets have been prepared during the development of FreqEasy-ML, with each collection stage improving the number of nodes, class balance, and relevance to the final room-condition classification task.

The first dataset contains 348 acoustic-feature rows across six room-condition and calibration labels, with approximately 58 samples per class. However, the measurements were collected from only one recording node, node04. This dataset was suitable for verifying the initial Edge Impulse import and training workflow, but it was not sufficient for final performance claims because the model could learn characteristics specific to one microphone, node, position, or recording session.

Initial dataset item	Current status
Total feature rows	348
Classes represented	6
Samples per class	Approximately 58

Node coverage	One node, node04
Intended use	Initial room-condition baseline and Edge Impulse workflow verification

The second dataset expanded the investigation to all four SDACS nodes and contains 1,123 clean samples across five controlled test-tone classes. Four classes contain 236 samples each. The 15 kHz class contains only 179 samples because node03 recorded two rows instead of the expected 59 during one capture.

Test-tone class	Rows	Status
test_tone_100hz	236	Complete
test_tone_500hz	236	Complete
test_tone_2khz	236	Complete
test_tone_5khz	236	Complete
test_tone_15khz	179	Incomplete

Although the second dataset provided a stronger frequency-classification baseline, its controlled tone labels were not representative of the final user-facing room conditions. The test-tone data remains useful for confirming that the firmware and classification pipeline can distinguish known frequency regions. However, a broader dataset was required to evaluate realistic acoustic conditions such as speech, environmental noise, quiet-room behaviour, and low-, mid-, or high-frequency emphasis.

The latest FreqEasy-ML dataset significantly improves on the earlier tone-based and single-node versions. A total of 11,426 rows were collected across six labels: low, mid, high, speech, noisy, and quiet-room/white-noise. After correcting mislabeled capture sessions, separating one continuous recording into two logical captures, removing two unmatched records, and trimming the beginning and end of each capture, 11,040 balanced node-level samples remained. Each label contains 1,840 samples collected across all four SDACS nodes.

The cleaned data was divided by complete capture session rather than by randomly selected individual rows. This produced 8,280 training rows and 2,760 testing rows while preventing samples from the same playback session from appearing in both sets. This approach reduces data leakage and provides a more realistic evaluation of how the model performs on an unseen acoustic capture.

Latest dataset item	Current status
Raw feature rows	11,426
Cleaned node-level rows	11,040
Classes represented	6
Samples per class	1,840
Node coverage	Four synchronized SDACS nodes
Training rows	8,280
Testing rows	2,760
Split method	Complete capture sessions

Intended use	Final room-condition ANN training and evaluation
---------------------	---

A room-level fused version of the latest dataset was also created by calculating the mean, standard deviation, and range of the four synchronized node measurements for each one-second interval. This produced 2,760 fused samples, with 460 samples per class. A preliminary Random Forest sanity check achieved approximately 23.8% accuracy using individual-node measurements and approximately 47.2% using the fused four-node data. Although these results are not the final Edge Impulse ANN results, they indicate that synchronized multi-node sensing provides more useful room-level classification information than treating each sensor independently.

9.2 ANN Training and Evaluation

The latest room-condition dataset is ready for feedforward ANN training in Edge Impulse. The preferred input is the four-node fused dataset because preliminary testing showed better class separation than the individual-node approach.

Model performance will be evaluated using:

- Training and validation accuracy
- Validation loss
- Test accuracy
- Confusion matrix
- Per-class precision and recall
- Model size and RAM requirements
- Raspberry Pi inference time

The network structure, feature selection, and class definitions may be adjusted if the results show:

- Overfitting
- Poor performance on unseen captures
- Confusion between speech and mid-frequency classes
- Confusion between noisy and frequency-dominant classes
- Excessive sensitivity to individual node responses

The earlier test-tone dataset will remain a controlled validation reference for known frequency regions. The broader room-condition dataset will be used to evaluate whether the model can classify realistic acoustic conditions. Only the trained Edge Impulse model, rather than the dataset itself, will be deployed to the Raspberry Pi.

Before ANN training, the value of distributed sensing was evaluated using a preliminary Random Forest model. The four synchronized node readings were fused by calculating the mean, standard deviation, and range of each retained feature for every one-second interval.

This produced:

- 2,760 room-level fused samples
- 460 fused samples per class
- Three training captures per class
- One testing capture per class

The preliminary results were:

- Individual-node accuracy: approximately 23.8%
- Four-node fused accuracy: approximately 47.2%

These values are not final Edge Impulse ANN results. However, they indicate that multi-node fusion provides more useful room-level information than isolated node measurements.

Final model development will prioritize:

- Four-node feature fusion
- Per-node level and frequency-response SPL calibration
- Clearer separation between class definitions
- Prediction averaging across a completed capture
- Confidence-threshold handling for uncertain classifications

9.3 Recommendation and Interface Integration

The final development stage is to connect the predicted ANN class to the treatment-recommendation logic and display the result through the SDACS interface on Flutter. End-to-end validation will be conducted to confirm that a recording can move from the ESP32-S3 node through MQTT, backend storage, feature processing, ANN classification, and finally to a clear recommendation in Flutter.

Recommendations based on the results which appear in flutter have yet to be decided or designed. The final report should document remaining limitations, including dataset size, low-frequency microphone accuracy, SPL calibration, and generalization to untested rooms.

10 CONCLUSIONS

FreqEasy-ML has progressed from an initial concept into a working machine-learning workflow supported by the SDACS platform. Four ESP32-S3 nodes can collect acoustic and environmental features, transmit measurements through MQTT, store data on the Raspberry Pi, and display matching telemetry through Flutter. The latest balanced dataset contains 11,040 cleaned node-level samples across six room-conditioned classes, together with 2,760 four-node fused samples. Preliminary testing showed that four-node fusion improved capture-held-out classification accuracy from approximately 23.8% to 47.2%, demonstrating measurable value from distributed sensing while identifying node calibration and class overlap as the main remaining limitations.

Final work will focus on Edge Impulse ANN training, Raspberry Pi deployment, capture-level prediction averaging, and Flutter recommendation integration.

11 REFERENCES

- [1] F. Scamoni, C. Scrosati, M. Depalma, and B. Barozzi, “Experimental evaluations of acoustic properties and long-term analysis of a novel indoor living wall,” *Journal of Building Engineering*, vol. 47, p. 103890, 2022, doi: 10.1016/j.jobbe.2021.103890.
- [2] S. Pradhan, G. Baig, W. Mao, L. Qiu, G. Chen, and B. Yang, “Smartphone-based Acoustic Indoor Space Mapping,” *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 2, no. 2, Art. 75, Jun. 2018, doi: 10.1145/3214278.
- [3] S. Jha, A. Gupta, A. Gupta, R. K. Mishra, and R. P. S. Jadon, “Analysis and mapping of the indoor acoustic environment in university study spaces,” *E3S Web of Conferences*, vol. 559, Art. 01007, 2024, doi: 10.1051/e3sconf/202455901007.
- [4] Sheng Chen, Renrui Tan, Zhiming Wang, Xinyu Tong, and Keqiu Li, “VoiceMap: Autonomous Mapping of Microphone Array for Voice Localization,” *IEEE Internet of Things Journal*, vol. 11, no. 2, pp. 2909–2922, Jan. 2024, doi: 10.1109/JIOT.2023.3294937.
- [5] J. Salamon and J. P. Bello, “Deep convolutional neural networks and data augmentation for environmental sound classification,” *IEEE Signal Processing Letters*, vol. 24, no. 3, pp. 279–283, Mar. 2017, doi: 10.1109/LSP.2017.2657381.
- [6] Google AI Edge, “LiteRT for Microcontrollers,” Google AI Edge Documentation. Accessed: Jun. 18, 2026. [Online]. Available: Google AI Edge LiteRT for Microcontrollers documentation.
- [7] Edge Impulse, “Edge Impulse Documentation,” Edge Impulse Documentation. Accessed: Jun. 18, 2026. [Online]. Available: Edge Impulse documentation.

APPENDICES

Appendix A: 4 Node Dataset Instructions

SDACS Amy MATLAB Handoff

Generated from: Amy_MATLAB_4_Node_Data_Dump.zip

Purpose

This folder gives Amy a clean starting point for MATLAB room-map plots. Use the summary files first, not the full raw per-second dataset.

Main file to use first

amy_matlab_roommap_summary_best_runs.csv

This file has one recommended 4-node summary set per label where possible. It includes x_m, y_m, z_metric, and z_value so Amy can make a quick scatter3 plot:

x = x_m

y = y_m

z = z_value

color/grouping = label or dominant_band_mode

Important files

1. amy_matlab_roommap_summary_best_runs.csv

Best capture summaries for MATLAB plotting. This is the easiest starting file.

2. amy_node_positions_template.csv

Placeholder node coordinates. Update x_m/y_m/height_m to the real room layout before final plots.

3. amy_matlab_summary_corrected_all_runs.csv

Full corrected summary table. 15 kHz target frequency metadata has been corrected from 5000 to 15000.

4. amy_matlab_column_dictionary.csv

Column meanings for the summary file.

Recommended MATLAB workflow

1. Import amy_matlab_roommap_summary_best_runs.csv.
2. Confirm rows, labels, and nodes.
3. Keep amy_node_positions_template.csv the same based on synthetic locations.
4. Join by node_id if you want to replace the placeholder coordinates.
5. Plot x_m, y_m, z_value for each label.
6. Repeat plots using z_metric / z_value or specific mean band ratio columns.

Suggested z-axis by label (Ignore this if you already have your own method)

- test_tone_100hz: mean_band_bass_ratio
- test_tone_500hz: mean_band_low_mid_ratio
- test_tone_2khz: mean_band_mid_ratio
- test_tone_5khz: mean_band_presence_ratio or mean_band_high_ratio
- test_tone_15khz: mean_band_high_ratio

Known notes

- SPL/db_spl is not calibrated yet. Treat it as a reference only.
- Band ratios are better for the first MATLAB plots.
- The 15 kHz target frequency metadata was wrong in the original exported files and has been corrected here.
- The summary file had a complete 15 kHz run that was not present in the raw acoustic feature CSV. The column `raw_rows_available_in_sdacs_acoustic_features_csv` shows this.
- About 59 rows means a healthy 60 s capture, since the firmware publishes about once per second.

Quick MATLAB starter

```
T = readtable("amy_matlab_roommap_summary_best_runs.csv", "TextType", "string");
figure;
scatter3(T.x_m, T.y_m, T.z_value, 80, T.z_value, "filled");
grid on;
xlabel("Room X (m)"); ylabel("Room Y (m)"); zlabel("Selected feature ratio");
title("SDACS 4-Node Acoustic Room Map");
colorbar;
text(T.x_m, T.y_m, T.z_value, T.node_id);
```

Instructions handling the data in edge impulse:

SDACS David Edge Impulse Handoff

Generated from: Amy_MATLAB_4_Node_Data_Dump.zip

Purpose

This folder gives David a cleaner Edge Impulse starting point. The training CSV removes metadata columns that could leak the answer or confuse the model.

Main file to use first

david_edge_impulse_training_features_clean.csv

This file contains:

- label: the class to predict
- numeric acoustic feature columns only

Use label as the class/output column. Use the remaining numeric columns as input features.

Do NOT train on these metadata columns

- target_freq_hz: this gives away the answer for frequency-label training.
- target_band: also gives away the answer.
- suggested_z_metric: helper for humans, not a sensor feature.
- node_id: useful for debugging but can cause the model to learn the node instead of the acoustic class.
- run_id, timestamp_iso, seq: collection metadata only.
- db_spl: uncalibrated for now.
- temp_c, rh_percent: environmental metadata, not acoustic class evidence.

Included files

1. david_edge_impulse_training_features_clean.csv

Best CSV to import into Edge Impulse for the first feature-based model.

2. david_edge_impulse_metadata_manifest.csv

One-to-one row manifest for debugging. Use row_index to trace a training row back to node_id/run_id/time.

3. david_edge_impulse_feature_dictionary.csv

Which columns are inputs, labels, or excluded metadata.

4. david_edge_impulse_label_balance_by_node.csv

Shows how many rows exist for each label and node.

5. david_edge_impulse_known_issues.csv

Known data issues and what was corrected.

Current label totals in the raw feature dump

```
-----
label rows
test_tone_100hz 236
test_tone_15khz 179 (failure in node03 capture)
test_tone_2khz 236
test_tone_500hz 236
test_tone_5khz 236
```

Current row balance by label/node

```
-----
node_id      node01 node02 node03 node04
label
test_tone_100hz 59  59  59  59
```

test_tone_500hz	59	59	59	59
test_tone_2khz	59	59	59	59
test_tone_5khz	59	59	59	59
test_tone_15khz	59	59	2	59

Important known issue

The original file had test_tone_15khz rows with target_freq_hz = 5000. That has been corrected to 15000 in the clean/metadata files.

Also, the raw acoustic feature dump has incomplete node03 coverage for test_tone_15khz: node03 has only 2 rows while the other nodes have 59. This is okay for a pilot, but it should be re-recorded for the final training dataset.

Recommended Edge Impulse approach

1. Start with david_edge_impulse_training_features_clean.csv.
2. Set label as the class label.
3. Confirm the remaining columns are numeric features.
4. Train a first classifier using the five labels:
 - test_tone_100hz
 - test_tone_500hz
 - test_tone_2khz
 - test_tone_5khz
 - test_tone_15khz
5. Check the confusion matrix.
6. If 15 kHz performs badly, re-record 15 kHz with all four nodes connected for the full 60 s.

Best interpretation

This is a feature-based model, not raw-audio training. The firmware has already extracted FFT/band/tone features, and Edge Impulse is learning from those numeric features.

Appendix B: Logbooks & Status Reports

WEEK 10 – AI DATASET COLLECTION AND MULTI-NODE FEATURE VALIDATION

Week: 10

Date Range: July 9–15, 2026

OBJECTIVES

- Begin structured AI dataset collection using SDACS nodes.
 - Validate simultaneous feature capture from multiple nodes.
 - Confirm Node-RED labeling workflow for machine-learning datasets.
 - Prepare exported data for MATLAB and Edge Impulse analysis.
-

ACTIVITIES COMPLETED

- Conducted initial AI dataset collection sessions using multiple SDACS nodes operating simultaneously.
- Validated collection of acoustic features including:
 - RMS
 - dBFS
 - Estimated dB SPL
 - FFT peak frequency
 - Low-, mid-, and high-frequency band energies
- Refined the Node-RED labeling workflow to support frequency-based dataset generation.

- Confirmed successful export of collected telemetry data into CSV format for post-processing and machine-learning applications.
 - Evaluated procedures for separating combined datasets into node-specific datasets for MATLAB and Edge Impulse workflows.
 - Verified stable communication between ESP32 nodes, MQTT, Node-RED, backend services, and data storage during collection sessions.
 - Continued review of feature quality and consistency across multiple sensing nodes.
-

MEETINGS & COMMUNICATION

- Coordinated dataset collection methodology with team members.
 - Discussed AI feature relevance, labeling strategy, and data preparation requirements.
 - Reviewed MATLAB and Edge Impulse integration plans for future model development.
-

DECISIONS MADE

- Use Node-RED as the primary AI dataset collection interface.
 - Continue collecting labeled frequency-response datasets before moving to model training.
 - Export both combined and node-specific datasets to support multiple analysis workflows.
 - Prioritize dataset quality and consistency over model development at this stage.
-

NEXT STEPS (WEEK 11)

- Expand dataset collection across additional frequency labels and acoustic conditions.
 - Import collected data into MATLAB for visualization and verification.
 - Prepare Edge Impulse-compatible datasets for initial model training.
 - Continue validating consistency between nodes during synchronized captures.
-

REFLECTION

Week 10 marked the transition from system integration toward machine-learning dataset generation. The SDACS platform successfully collected labeled acoustic feature data from multiple nodes, demonstrating readiness for AI development and future model training. The

primary focus moving forward will be expanding the dataset, validating feature quality, and preparing data for MATLAB and Edge Impulse analysis.

END OF LOGBOOK ENTRY

WEEK 9 – FLUTTER–BACKEND VALIDATION, REMOTE ACCESS, AND TEST PLAN DEVELOPMENT

Week:

9

Date Range: June 28–July 8, 2026

OBJECTIVES

- Complete Flutter-to-backend integration testing.
- Validate remote Raspberry Pi access using Tailscale.
- Finalize frontend/backend integration testing documentation.
- Prepare the system for AI dataset collection.

ACTIVITIES COMPLETED

- Verified Flutter communication with the Raspberry Pi backend through REST API endpoints.
- Updated the backend configuration to use the correct default IP address and confirmed live telemetry within Flutter.
- Configured Tailscale remote access and documented the setup process for teammates.
- Expanded the Frontend–Backend Integration Test Plan with backend verification steps, validation procedures, and recommended figures.
- Reviewed MATLAB integration workflow for future AI analysis using exported acoustic features.
- Confirmed backend readiness for remote testing and continued end-to-end system validation.

MEETINGS & COMMUNICATION

- Coordinated backend integration and remote-access workflow with teammates.
- Reviewed documentation, testing priorities, and integration milestones.

DECISIONS MADE

- Standardize Tailscale for remote backend access.
- Use REST APIs as the primary interface between Flutter and the backend.
- Continue validating the complete firmware-to-Flutter pipeline before dataset collection.

NEXT STEPS (WEEK 10)

- Complete end-to-end validation across firmware, backend, and Flutter.
- Begin collecting labelled AI datasets.
- Finalize remaining UI improvements and report figures.

REFLECTION

The past two weeks focused on validating the SDACS software stack rather than adding new features. Reliable Flutter connectivity, remote backend access, and structured integration testing significantly improved the project's readiness for collaborative development, AI dataset collection, and final system validation.

Project Status Report – Week 10

2026-07-08

Project Name: Smart Distributed Acoustic Calibration System (SDACS) **Team Members:** Kyle Dick, David Toma, Amy Wentzell**Project Overall Status:** GREEN/YELLOW/RED**Status Overview:** Assessment 2 items were completed in this weeks status report.

Activity Title with start date and also planned completion date and name of responsible team member	Status (Last Week) G/Y/R	Status (This Week)		Plan (Next Week)
		Plan (as planned last week for this week)	Actual Green/Yellow/Red	
Firmware Development and Backend Integration with Flutter 2026/06/30 – 2026/07/07 Kyle Dick	G	The bug fixes needed to fix the microphone quality is due to a data-storing issue for the specific microphone IC. By debugging the firmware, solid and clean datasets will be finalized for NodeRED to display on the website.	G	Update the flutter UI to include additional buttons and features
	G	The software test plan will be written to verify that the website is	G	Completed.

Software Test Plan 2026/06/30 – 2026/07/015 Kyle Dick		properly connected to the backend (NodeRED) and verify basic functional features such as completion of an audio capture.		
Edge Impulse AI Model Development and Testing 2026/06/30 – 2026/07/0\15 David Toma	G	The AI model will be developed and tested using Edge Impulse and data from the SDACS nodes.	G	Train with 4 nodes of data over multiple frequency bands.
3-D Modelling Development in MATLAB 2026/06/30 – 2026/07/015 Amy Wentzell & Kyle Dick	G	The basic algorithm to use the frequency ratios/percentages to display the results to the user will be developed in MATLAB as a prototype.	G	First revision is developed; further testing is required and integration with the Flutter app. Development has continued to include 4 nodes of data from KD.

Guidelines:

- Overall Status: Green is the best, Red is the worst, and Yellow is in between. You may add + or – suffix to indicate the status more accurately.
- Planned completion date and name of responsible team member: indicate if these are unchanged or changed/new.
- You must compare your status in three ways:
 1. Status of this week versus the status in last week
 2. Status of this week versus the plan for this week
 3. Planned completion date and responsible person (this week) versus the planned completion date and responsible person (last week)

Project Status Report – Week 9

2026-07-08

Project Name: Smart Distributed Acoustic Calibration System (SDACS) Team Members: Kyle Dick, David Toma, Amy Wentzell

Project Overall Status: GREEN/YELLOW/RED

Status Overview: Kyle has made advances in improving the firmware and integrating the back-end NodeRED to the Flutter App. David has developed and tested an Edge Impulse model to improve suggestions for the user and the available analysis through Flutter. Amy has researched and began developing the MATLAB algorithm for graphing the results using a 3-D model on the Flutter app.

Activity Title with start date and also planned completion date and name of responsible team member	Status (Last Week) G/Y/R	Status (This Week)		Plan (Next Week)
		Plan (as planned last week for this week)	Actual Green/Yellow/Red	
Firmware Development and Backend Integration with Flutter 2026/06/30 – 2026/07/07 Kyle Dick	G	The bug fixes needed to fix the microphone quality is due to a data-storing issue for the specific microphone IC. By debugging the firmware, solid and clean datasets will be finalized for Node-RED to display on the website.	G	Frequency data has been verified to be accurate, but SPL calibration is still needed. SPL calibration will be performed next week to adjust the dB offset for all 4-5 nodes.
Edge Impulse AI Model Development and Testing	G	The AI model will be developed and tested using Edge Impulse and data from the SDACS nodes.	G	Continue testing. Collect more data to improve accuracy for different test cases (etc. low, mid, high freq).

2026/06/30 – 2026/07/07				
David Toma				

Guidelines:

- Overall Status: Green is the best, Red is the worst, and Yellow is in between. You may add + or – suffix to indicate the status more accurately.
- Planned completion date and name of responsible team member: indicate if these are unchanged or changed/new.
- You must compare your status in three ways:
 1. Status of this week versus the status in last week
 2. Status of this week versus the plan for this week
 3. Planned completion date and responsible person (this week) versus the planned completion date and responsible person (last week)

Appendix C: Feature Dictionary for Dataset 3

column	role	description
label	label	Edge Impulse classification target.
capture_id	metadata/group	Logical capture ID. Keep complete groups together during validation.
node_id	metadata	Physical node identifier; do not use as an acoustic feature.
sample_index	metadata	One-second index within the logical capture.
dbfs	model_feature	Relative acoustic level. Kept instead of rms and uncalibrated db_spl.
f_peak_acoustic_hz	model_feature	Dominant peak within the firmware acoustic search range.
low_rumble_ratio	model_feature	Relative low-frequency/rumble content.
band_bass_ratio	model_feature	Energy ratio in the bass band.
band_low_mid_ratio	model_feature	Energy ratio in the low-mid band.
band_mid_ratio	model_feature	Energy ratio in the mid band.
band_presence_ratio	model_feature	Energy ratio in the presence band.
band_high_ratio	model_feature	Energy ratio in the high band.
band_bass_peak_hz	model_feature	Peak frequency inside the bass band.
band_low_mid_peak_hz	model_feature	Peak frequency inside the low-mid band.
band_mid_peak_hz	model_feature	Peak frequency inside the mid band.
band_presence_peak_hz	model_feature	Peak frequency inside the presence band.
band_high_peak_hz	model_feature	Peak frequency inside the high band.
dominant_band_ratio	model_feature	Energy ratio of the strongest firmware band.
dominant_band_peak_hz	model_feature	Peak frequency of the strongest firmware band.
fft_low_ratio	model_feature	Broad low-frequency FFT energy ratio.
fft_mid_ratio	model_feature	Broad mid-frequency FFT energy ratio.
fft_high_ratio	model_feature	Broad high-frequency FFT energy ratio.
fft_total_energy	model_feature	Total FFT energy for the one-second feature window.
dbfs_mean	room_fused_model_feature	Across-node mean of dbfs for one aligned second.
dbfs_std	room_fused_model_feature	Across-node std of dbfs for one aligned second.
dbfs_range	room_fused_model_feature	Across-node range of dbfs for one aligned second.
f_peak_acoustic_hz_mean	room_fused_model_feature	Across-node mean of f_peak_acoustic_hz for one aligned second.
f_peak_acoustic_hz_std	room_fused_model_feature	Across-node std of f_peak_acoustic_hz for one aligned second.
f_peak_acoustic_hz_range	room_fused_model_feature	Across-node range of f_peak_acoustic_hz for one aligned second.
low_rumble_ratio_mean	room_fused_model_feature	Across-node mean of low_rumble_ratio for one aligned second.
low_rumble_ratio_std	room_fused_model_feature	Across-node std of low_rumble_ratio for one aligned second.

low_rumble_ratio_range	room_fused_model_feature	Across-node range of low_rumble_ratio for one aligned second.
band_bass_ratio_mean	room_fused_model_feature	Across-node mean of band_bass_ratio for one aligned second.
band_bass_ratio_std	room_fused_model_feature	Across-node std of band_bass_ratio for one aligned second.
band_bass_ratio_range	room_fused_model_feature	Across-node range of band_bass_ratio for one aligned second.
band_low_mid_ratio_mean	room_fused_model_feature	Across-node mean of band_low_mid_ratio for one aligned second.
band_low_mid_ratio_std	room_fused_model_feature	Across-node std of band_low_mid_ratio for one aligned second.
band_low_mid_ratio_range	room_fused_model_feature	Across-node range of band_low_mid_ratio for one aligned second.
band_mid_ratio_mean	room_fused_model_feature	Across-node mean of band_mid_ratio for one aligned second.
band_mid_ratio_std	room_fused_model_feature	Across-node std of band_mid_ratio for one aligned second.
band_mid_ratio_range	room_fused_model_feature	Across-node range of band_mid_ratio for one aligned second.
band_presence_ratio_mean	room_fused_model_feature	Across-node mean of band_presence_ratio for one aligned second.
band_presence_ratio_std	room_fused_model_feature	Across-node std of band_presence_ratio for one aligned second.
band_presence_ratio_range	room_fused_model_feature	Across-node range of band_presence_ratio for one aligned second.
band_high_ratio_mean	room_fused_model_feature	Across-node mean of band_high_ratio for one aligned second.
band_high_ratio_std	room_fused_model_feature	Across-node std of band_high_ratio for one aligned second.
band_high_ratio_range	room_fused_model_feature	Across-node range of band_high_ratio for one aligned second.
band_bass_peak_hz_mean	room_fused_model_feature	Across-node mean of band_bass_peak_hz for one aligned second.
band_bass_peak_hz_std	room_fused_model_feature	Across-node std of band_bass_peak_hz for one aligned second.
band_bass_peak_hz_range	room_fused_model_feature	Across-node range of band_bass_peak_hz for one aligned second.
band_low_mid_peak_hz_mean	room_fused_model_feature	Across-node mean of band_low_mid_peak_hz for one aligned second.
band_low_mid_peak_hz_std	room_fused_model_feature	Across-node std of band_low_mid_peak_hz for one aligned second.
band_low_mid_peak_hz_range	room_fused_model_feature	Across-node range of band_low_mid_peak_hz for one aligned second.
band_mid_peak_hz_mean	room_fused_model_feature	Across-node mean of band_mid_peak_hz for one aligned second.
band_mid_peak_hz_std	room_fused_model_feature	Across-node std of band_mid_peak_hz for one aligned second.
band_mid_peak_hz_range	room_fused_model_feature	Across-node range of band_mid_peak_hz for one aligned second.

band_presence_peak_hz_mean	room_fused_model_feature	Across-node mean of band_presence_peak_hz for one aligned second.
band_presence_peak_hz_std	room_fused_model_feature	Across-node std of band_presence_peak_hz for one aligned second.
band_presence_peak_hz_range	room_fused_model_feature	Across-node range of band_presence_peak_hz for one aligned second.
band_high_peak_hz_mean	room_fused_model_feature	Across-node mean of band_high_peak_hz for one aligned second.
band_high_peak_hz_std	room_fused_model_feature	Across-node std of band_high_peak_hz for one aligned second.
band_high_peak_hz_range	room_fused_model_feature	Across-node range of band_high_peak_hz for one aligned second.
dominant_band_ratio_mean	room_fused_model_feature	Across-node mean of dominant_band_ratio for one aligned second.
dominant_band_ratio_std	room_fused_model_feature	Across-node std of dominant_band_ratio for one aligned second.
dominant_band_ratio_range	room_fused_model_feature	Across-node range of dominant_band_ratio for one aligned second.
dominant_band_peak_hz_mean	room_fused_model_feature	Across-node mean of dominant_band_peak_hz for one aligned second.
dominant_band_peak_hz_std	room_fused_model_feature	Across-node std of dominant_band_peak_hz for one aligned second.
dominant_band_peak_hz_range	room_fused_model_feature	Across-node range of dominant_band_peak_hz for one aligned second.
fft_low_ratio_mean	room_fused_model_feature	Across-node mean of fft_low_ratio for one aligned second.
fft_low_ratio_std	room_fused_model_feature	Across-node std of fft_low_ratio for one aligned second.
fft_low_ratio_range	room_fused_model_feature	Across-node range of fft_low_ratio for one aligned second.
fft_mid_ratio_mean	room_fused_model_feature	Across-node mean of fft_mid_ratio for one aligned second.
fft_mid_ratio_std	room_fused_model_feature	Across-node std of fft_mid_ratio for one aligned second.
fft_mid_ratio_range	room_fused_model_feature	Across-node range of fft_mid_ratio for one aligned second.
fft_high_ratio_mean	room_fused_model_feature	Across-node mean of fft_high_ratio for one aligned second.
fft_high_ratio_std	room_fused_model_feature	Across-node std of fft_high_ratio for one aligned second.
fft_high_ratio_range	room_fused_model_feature	Across-node range of fft_high_ratio for one aligned second.
fft_total_energy_mean	room_fused_model_feature	Across-node mean of fft_total_energy for one aligned second.
fft_total_energy_std	room_fused_model_feature	Across-node std of fft_total_energy for one aligned second.
fft_total_energy_range	room_fused_model_feature	Across-node range of fft_total_energy for one aligned second.